Lesson 2

Container & Exercises

Haoxiang Sun @ RUC CS



Higher-Order Function

demo

```
def make_adder(k):
    def adder(n):
        return n + k
    return adder

func = make_adder(3)
func(4)
```

Lambda Function

demo

```
def make_adder(k):
    return lambda n: n + k

func = make_adder(3)
func(4)
```

Example

```
def print_sums(n):
    print(n)
    def next_sum(k):
        return print_sums(n+k)
    return next_sum

print_sums(1)(3)(5)
```

Recursion

```
def skip_factorial(n):
    """Return the product of positive integers n * (n - 2) * (n - 4) * ...

    >>> skip_factorial(5) # 5 * 3 * 1
    15
    >>> skip_factorial(8) # 8 * 6 * 4 * 2
    384
    """
    if ___:
        return ___
    else:
        return n * skip_factorial(n - 2)
```

List 列表

demo

```python
lst = ["Hello", True, "this", "is", "Python", 3.13]
lst2 = [1, 2, 3, 4, 5, 6]
print(lst[3])
print(lst[-1])
print(lst[1:3])
print(lst[-1:-4:-1])
print(lst[-1:-4:-2] + lst2[:3])

lst.append("more!")
print(lst)
del lst[3]
print(lst)
print(len(lst))

print(["No"] * 4)
print("this" in lst)
```

List 列表

demo

```python
lst = ["Hello", True, "this", "is", "Python", 3.13, "is"]
for elem in lst:
    print(elem)

for idx in range(len(lst)):
    print(lst[idx])

def print_lst(lst):
    for idx, elem in enumerate(lst):
        print(f"The {idx}th elem in lst is {elem}")

print_lst(lst)
lst.reverse()
print_lst(lst)

print(lst.count("is"))

lst2 = [-3.123, 3, 3.4, -100]
print(max(lst2))
```

List 列表

demo

```
lst = [[3, -4], [9, -8], [10, -7], [2, -5]]

print(sorted(lst))
print(sorted(lst, key=lambda x: x[0]))
print(sorted(lst, key=lambda x: x[0] * x[1]))
print(sorted(lst, key=lambda x: x[1], reverse=True))
```

List Comprehension

```
lst = [[3, -4], [9, -8], [10, -7], [2, -5]]

lst2 = [[_[0], _[1] + 1] for _ in lst if _[0] % 2 == 1]
```

Tuple 元组

- 元组可以理解为不可修改的列表
- (1, 2, 3, 4, 5)

- 基于此，元组是可以被Hash的，因此他可以作为字典的 key

- Mutable & immutable.

Dictionary 字典

- 键值对

demo

```python
dic = {"Alex": [19, 1], "Bob": [20, 1], "Alice": [17, 0]}

print(dic["Alex"][1])
print(dic["Bob"])
print(dic["John"])
print("Alice" in dic)
# insert or assign
dic["Alex"] = [20, 1]
dic["John"] = [22, 0]
del dic["Bob"]

print(dic.items())
print(dic.keys())
print(dic.values())
dic.update({"Peter": [23, 1]})
print(dic)
```

Set 集合

demo

```
st = {10, 3, 2, 4, 5}

print(st)
print(5 in st)
print(6 in st)
st.add(6)
print(6 in st)
st.add(5)
print(st)
# Check if exists!
st.remove(5)
print(st)

st1 = {1, 2, 3, 4, 5}
st2 = {2, 3, 5, 6, 7}
print(st1 | st2)
print(st1 & st2)
print(st1 - st2)
print(st2 - st1)
print(st1 ^ st2)
```

练习题 1

- 不使用电脑，分析以下程序的输出

magic

```
lst = [[]] * 3
lst[0].append("This is a[0][0]")
lst[1].append("This is a[1][0]")
lst[2].append("This is a[2][0]")
print(lst)
```

练习题 2

- 学生成绩查询系统
- 输入姓名 + 平时成绩 + 期末成绩
- Alex 60 90
- Bob 80 75
- 最终成绩为 0.8 * 平时成绩 + 0.2 * 期末成绩
- 接下来有若干查询
- Q1: 按照期末成绩降序排列，同分者按姓名字典序升序排列
- Q2: 按照最终成绩降序排列，同分者按姓名字典序升序排列
- Q3: 按照最终成绩降序排列，同分者按期末成绩降序排列
- Name(例如 Alice): 查询某个人的信息，假如这个人不在成绩单中，输出 “Alice 没有选课”
- 假如在成绩单中，输出 “Alice 的平时成绩为 90，期末成绩为 80，总评成绩为 88”
- insert Alice 50 60: 插入一个学生，假如他已经在成绩单中，输出“错误，Alice的成绩已录入”，否则更新数据表

练习题 3

- 词频统计：给定英文字符串，进行词频统计和字母频率统计. 词频统计按字典序排序，字母统计按频率排序
- 单词频率统计:
- apple: 2
- banana: 3
- pear: 2
- 字母频率统计:
- d: 3
- a: 1
- b: 1
- c: 1